

UNIVERSIDAD EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

Organización de Computadores

Parcial 1 — Documento de Estrategia

ALU de 32 bits y ALU de tres entradas sobre Nand2Tetris

Integrantes

Santiago Guerra Vásquez
Emmanuel Cardona Castañeda
Jacobó Restrepo Torres

Septiembre de 2026

Introducción

Este documento reúne la estrategia que seguimos para resolver las dos preguntas del parcial. En los dos casos decidimos no partir de cero: usamos la ALU.hdl de 16 bits que ya habíamos construido y probado durante el curso como punto de partida, y desde ahí extendimos el diseño para cada problema. Nos pareció la manera más segura de trabajar, porque cualquier error nuevo que metiéramos iba a estar en la parte que agregábamos, no en la aritmética de base que ya sabíamos que funcionaba.

Las dos secciones siguen la misma estructura: primero explicamos la estrategia y las decisiones de diseño, y cerramos con cómo probamos que el chip funciona. La segunda pregunta agrega además el análisis de qué tan práctico es este tipo de diseño en un procesador real.

Pregunta 1 — ALU de 32 bits

1.1 Estrategia general

La idea central de ALU32 es partir cada operando de 32 bits en dos mitades de 16 bits — xLow/xHigh para X, yLow/yHigh para Y — y aplicarles la misma lógica de la ALU del curso a las dos mitades en paralelo, con los mismos seis bits de control (zx, nx, zy, ny, f, no) para ambas mitades a la vez. No creamos un segundo juego de bits de control para la mitad alta, porque el enunciado pide una ALU de 32 bits: una sola operación sobre un operando más ancho, no dos operaciones independientes de 16 bits compartiendo la misma instrucción.

1.2 Por qué el AND no necesita nada especial y la suma sí

Poner una mitad en cero o negarla no depende de la otra mitad: NOT de 32 bits es exactamente NOT de la mitad baja seguido de NOT de la mitad alta, y lo mismo pasa con anularla. Por eso los pasos 1 y 2 del HDL (procesar X y procesar Y) se hacen mitad por mitad sin ningún problema, igual que el AND del paso 3: cada bit del resultado depende solo del bit correspondiente de X y de Y, así que no hay ninguna interacción entre Low y High.

La suma es distinta. Si xLow + yLow se pasa de 16 bits, ese desborde tiene que sumarse a xHigh + yHigh, porque si no el resultado de 32 bits queda mal. Ese es el problema nuevo que no existe en la ALU de 16

bits del curso: hay que calcular y propagar un acarreo entre las dos mitades.

1.3 Cómo se detecta el acarreo entre mitades

El problema es que Add16, tal como está definido en Nand2Tetris, no expone ningún acarreo de salida — solo entrega el resultado de 16 bits. Para reconstruirlo usamos la fórmula clásica de generar/propagar un acarreo, a partir de los bits más significativos de los dos operandos y del bit más significativo de la suma: hay generación de acarreo cuando los dos bits 15 de entrada son 1, y hay propagación cuando al menos uno de los dos lo es; el acarreo que sale de la mitad baja es la generación o, en su defecto, la propagación combinada con que la suma en esa posición haya dado 0.

La lógica es: si los dos bits 15 son 1, seguro hay acarreo (se generó uno nuevo). Si solo uno de los dos es 1, hay acarreo únicamente si la suma en esa posición dio 0, es decir, si se propagó un acarreo que venía de más abajo. Ese acarreo ya calculado se mete como un bit extra en la suma de la mitad alta, junto con xHigh y yHigh.

1.4 Salida final: no, zr y ng

El bit no funciona igual que en la ALU de 16 bits, aplicado a cada mitad de la salida por separado. La bandera ng sale prácticamente gratis: como se trabaja en complemento a dos, el bit de signo de un número de 32 bits es el bit más alto de la mitad High, así que ng es directamente ese bit. La bandera zr sí pide más trabajo, porque hay que revisar los 32 bits del resultado completo: partimos la salida en cuatro bloques de 8 bits, usamos Or8Way en cada uno para saber si tiene algún 1, y combinamos esas cuatro respuestas con OR — si ninguna tiene un 1, el resultado completo es cero.

1.5 Validación

Para probar ALU32 armamos ALU32.tst con 144 casos: las 18 funciones clásicas de la ALU de 16 bits aplicadas a 8 pares de X,Y distintos, elegidos a propósito para forzar casos donde el acarreo cruza de la mitad baja a la alta (por ejemplo 0x0000FFFF + 1) y casos con el bit de signo en el límite (0x7FFFFFFF + 1). Los valores esperados no salieron de correr el simulador y confiar ciegamente en el resultado: los calculamos aparte en Python, replicando la aritmética de 32 bits bit a bit, y después comparamos ese cálculo contra la salida real del Hardware Simulator. Los 144 casos coincidieron exactamente; el archivo ALU32.cmp que queda en la entrega es justamente esa comparación (Comparison ended successfully).

Pregunta 2 — ALU de tres entradas

2.1 Estrategia general

Para la ALU3 la decisión de diseño fue distinta a la de ALU32. En vez de armar una red de compuertas nueva que reciba tres operandos a la vez, usamos dos veces la ALU de 16 bits que ya existía en el curso (ALU.hdl), una después de la otra: la primera combina X e Y y produce un resultado intermedio w; la segunda combina w con Z. Cada etapa usa el mismo patrón de control que ya conocíamos (zx, nx, zy, ny, f, no), solo que la segunda etapa tiene su propio juego de cuatro bits (zz, nz, fz, noz) para decidir qué le hace a Z y qué función aplica entre w y Z.

2.2 Por qué w entra sin zx/nx propios

La entrada w de la segunda ALU se conecta con zx=false y nx=false fijos, en vez de darle su propio par de bits. La razón es que w ya sale completamente procesado de la primera etapa: ponerlo en cero o negarlo otra vez ahí no agrega ninguna operación que no se pudiera lograr ya ajustando los bits de la

etapa 1. A cambio de esa simplificación, la ALU3 queda con 10 bits de control en total en vez de 12. La consecuencia es que, cuando hace falta negar w antes de combinarlo con Z (por ejemplo, para restar Z), esa negación se pide en la etapa 1 con no=1 y se deshace al final con noz=1: $x + y - z = !(x + y) + z$.

2.3 Conjunto de operaciones y bits de control

Con 6 bits para la etapa 1 y 4 para la etapa 2 hay $2^{10} = 1024$ combinaciones posibles. Para este documento elegimos las que nos parecieron más representativas de lo que puede hacer una ALU de tres entradas en la práctica:

Operación	zx nx zy ny f no	zz nz fz noz
$x + y + z$	0 0 0 0 1 0	0 0 1 0
$(x \text{ AND } y) + z$	0 0 0 0 0 0	0 0 1 0
$(x + y) \text{ AND } z$	0 0 0 0 1 0	0 0 0 0
$x + y - z$	0 0 0 0 1 1	0 0 1 1
$x - y + z$	0 1 0 0 1 1	0 0 1 0
$x + y$ (z ignorado)	0 0 0 0 1 0	1 0 1 0
$x + y + 1$	0 0 0 0 1 1	1 1 1 1
$!(x + y + z)$	0 0 0 0 1 0	0 0 1 1
$(x + y) - 1$	0 0 0 0 1 0	1 1 1 0

2.4 Validación

El archivo ALU3.tst tiene 42 casos: tomamos 6 funciones representativas de la etapa 1 ($x+y$, $x-y$, $x \text{ AND } y$, $-x$, $x+1$, y la función que da 0) y las emparejamos con 6 de la etapa 2 ($w+z$, $z-w$, $w \text{ AND } z$, z anulado, $!(w+z)$, y z ignorado); cada pareja se probó sobre 7 combinaciones de x , y , z que incluyen los tres operandos en cero, los tres en -1 (0xFFFF), overflow con signo al encadenar dos sumas ($32767+1+1$) y el valor más negativo de 16 bits sumado a operandos chicos. Igual que con ALU32, cada valor esperado se calculó aparte en Python antes de correr el simulador, y los 42 coincidieron (Comparison ended successfully).

2.5 Análisis de arquitectura: ¿tiene sentido fabricar esto de verdad?

Nuestra respuesta corta es que no, al menos no como instrucción de propósito general de un solo ciclo, aunque el ejercicio sí deja ver en qué contexto sí tendría sentido.

El primer problema aparece en el set de instrucciones. Una ALU de dos entradas se controla con 6 bits, que caben sin apretar en el opcode de una instrucción de 16 bits como las de Hack. La ALU3 necesita 10 bits de control más una tercera dirección de registro fuente (X, Y y Z, en vez de solo X e Y). Eso ya no entra en una instrucción de 16 bits sin quitarle espacio a la parte que direcciona memoria o registros, así que en la práctica obligaría a usar instrucciones más largas (32 bits) o a partir la operación en dos palabras — una de control y otra de operandos —, lo que complica bastante el decodificador de instrucciones.

El segundo problema es el consumo de energía. El camino de datos de la ALU3 son dos ALUs completas de 16 bits en serie, así que cada operación mueve aproximadamente el doble de compuertas que una

ALU normal de dos entradas. Eso pasa incluso cuando Z no hace falta ($zz=1$): la segunda ALU igual procesa $w+0$ puerta por puerta, porque el hardware no tiene forma de saltarse la etapa si no se necesita.

El tercer problema es la unidad de control y el camino crítico. No solo hay que decodificar 10 señales en vez de 6; la salida w de la primera etapa es la entrada de la segunda, así que la señal tiene que atravesar dos sumadores completos en cascada antes de estabilizarse. Eso alarga el tiempo de propagación de la ALU entera, y en un diseño síncrono normal eso se traduce en bajar la frecuencia de reloj o partir la operación en dos ciclos con un registro guardando w a la mitad, lo que a su vez pide más lógica de control.

Juntando los tres puntos, no nos parece que valga la pena como instrucción genérica: el costo en área, energía y camino crítico no se justifica frente a simplemente ejecutar dos instrucciones normales de la ALU de dos entradas seguidas ($w = x \text{ op } y$, y después $\text{out} = w \text{ op } z$), que es literalmente lo que la ALU3 hace por dentro, solo que fusionado en una sola instrucción. Donde sí le vemos sentido es como instrucción fusionada especializada — al estilo de un fused multiply-add — dentro de una unidad vectorial o un DSP, donde combinar tres operandos es tan frecuente (acumuladores, filtros FIR) que amortizar ese costo extra en una unidad dedicada, fuera del pipeline de propósito general, sí compensa.

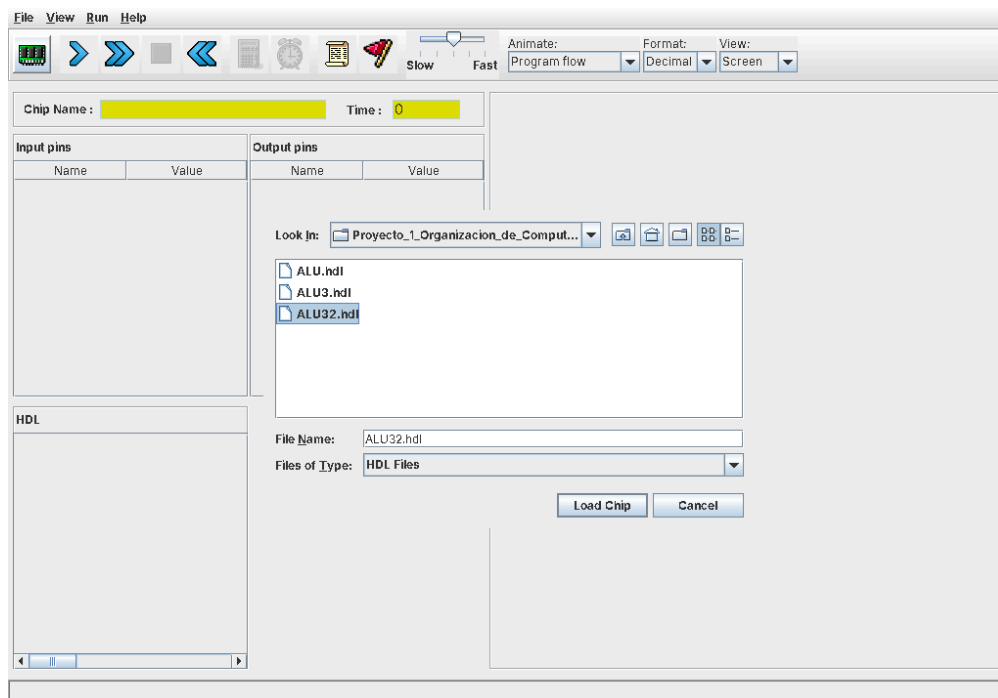
Cómo usar las ALUs en el Hardware Simulator

Esta sección explica, con capturas tomadas directamente del Hardware Simulator de Nand2Tetris, el procedimiento que seguimos para cargar y evaluar las dos ALUs de este documento. La idea es que cualquiera pueda repetir las pruebas sin tener que adivinar los pasos.

3.1 Cargar un chip

El simulador se abre vacío. Para probar un circuito hay que ir al menú File y escoger "Load Chip...", lo que abre un explorador de archivos limitado a archivos .hdl. Ahí se navega hasta la carpeta del proyecto (en nuestro caso la carpeta que contiene ALU.hdl, ALU32.hdl y ALU3.hdl) y se hace doble clic sobre el archivo que se quiere simular. El simulador compila el HDL en ese momento y, si no hay errores, deja el chip listo para evaluar.

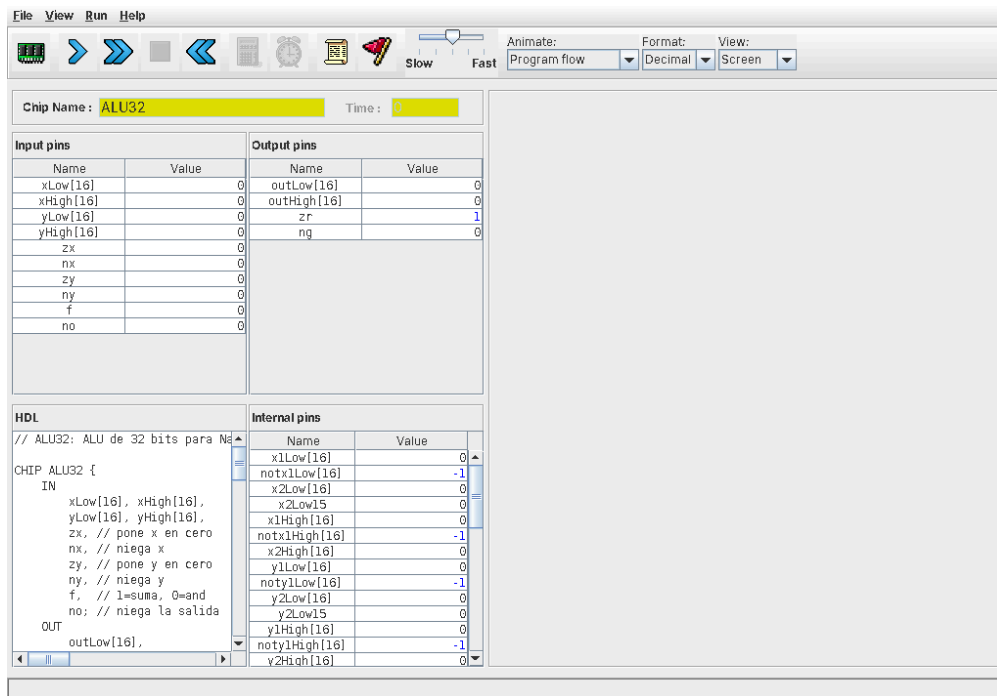
Figura 1. Diálogo "Load Chip" mostrando los archivos .hdl del proyecto.



3.2 Panel de trabajo: pines y HDL

Una vez cargado un chip, la ventana se organiza en cuatro paneles: arriba a la izquierda los Input pins (donde se escriben los valores de entrada), arriba a la derecha los Output pins (donde aparece el resultado después de evaluar), abajo a la izquierda el código HDL del chip cargado (de solo lectura, sirve para confirmar que se cargó el archivo correcto) y abajo a la derecha los Internal pins, que muestran el valor de las señales intermedias de la implementación — por ejemplo x2Low, addHighBase o carryFromLow en el caso de ALU32. Este último panel es el que más usamos para verificar que la propagación de acarreo funcionara como se esperaba, y no solo el resultado final.

Figura 2. ALU32 recién cargada, antes de evaluar: todas las entradas en cero y zr en 1.



3.3 Asignar entradas y evaluar

Para fijar un valor se hace doble clic sobre la celda de la columna Value, en la fila del pin correspondiente, se borra el contenido y se escribe el número en decimal (el simulador acepta enteros con signo entre -32768 y 32767 para un bus de 16 bits). Después de escribir todos los valores de entrada que se quieren probar, se evalúa el circuito desde el menú Run, opción "Eval" (equivalente a pulsar el ícono del rayo en la barra de herramientas). Si el chip tiene errores de HDL o referencia partes sin resolver, esta opción aparece deshabilitada; si todo está bien, al hacer clic el simulador recalcula instantáneamente todos los pines, tanto de salida como internos.

3.4 Ejemplo con ALU32: propagación de acarreo entre mitades

Para comprobar en la práctica el mecanismo descrito en la sección 1.3, cargamos ALU32 y fijamos xLow=20000, yLow=-10000, con f=1 (modo suma) y el resto de las señales de control en 0. El resultado, visible tanto en los Output pins como en los Internal pins, confirma el comportamiento esperado: outLow queda en 10000 y outHigh queda en 1, es decir, la suma de las dos mitades bajas produjo un acarreo que subió hasta la mitad alta, exactamente lo que calcula la lógica de generate/propagate descrita antes, sin que xHigh ni yHigh tuvieran que aportar nada.

Figura 3. ALU32 evaluada con $x_{Low}=20000$, $y_{Low}=-10000$, $f=1$: se observa el acarreo propagado hacia outHigh.

The screenshot shows the ALU32 evaluation interface. The top bar includes menu items (File, View, Run, Help) and a toolbar with simulation controls (Play, Step, Pause, Stop, Fast, Slow) and settings (Animate: Program flow, Format: Decimal, View: Screen). The main window displays the following data:

Input pins		Output pins	
Name	Value	Name	Value
xLow[16]	20000	outLow[16]	10000
xHigh[16]	0	outHigh[16]	1
yLow[16]	-10000	zr	0
yHigh[16]	0	ng	0
zx	0		
nx	0		
zy	0		
ny	0		
f	1		
no	0		

Internal pins	
Name	Value
andHigh[16]	0
addLow[16]	10000
addLow15	0
generateCarry	0
propagateCarry	1
notAddLow15	1
propagatedCarry	1
carryFromLow	1
addHighBase[16]	0
addHigh[16]	1
foutLow[16]	10000
foutHigh[16]	1
notfoutLow[16]	-10001
outByte0[8]	16

HDL

```
// ALU32: ALU de 32 bits para Nda
CHIP ALU32 {
    IN
        xLow[16], xHigh[16],
        yLow[16], yHigh[16],
        zx, // pone x en cero
        nx, // niega x
        zy, // pone y en cero
        ny, // niega y
        f, // 1=suma, 0=and
        no; // niega la salida
    OUT
        outLow[16],

```

3.5 Ejemplo con ALU3: combinación de tres operandos

De la misma forma, cargamos ALU3 y fijamos $x=5$, $y=3$, $z=2$, con $f=1$ y $fz=1$ (suma en ambas etapas) y el resto de las señales de control en 0. El panel de Internal pins muestra el resultado intermedio $w=8$ (la suma de x con y , calculada por la primera instancia de ALU), y los Output pins muestran $out=10$, que es justamente $w+z=8+2$, calculado por la segunda instancia. Esto ilustra de forma directa la estrategia de encadenar dos ALUs descrita en la sección 2.1: el resultado final es literalmente la salida de la segunda ALU, que a su vez depende de la salida de la primera.

Figura 4. ALU3 evaluada con $x=5$, $y=3$, $z=2$, $f=1$, $fz=1$: $w=8$ internamente y $out=10$ como resultado final.

The screenshot shows the ALU3 evaluation interface. The top bar includes menu items (File, View, Run, Help) and a toolbar with simulation controls (Play, Step, Pause, Stop, Fast, Slow) and settings (Animate: Program flow, Format: Decimal, View: Screen). The main window displays the following data:

Input pins		Output pins	
Name	Value	Name	Value
x[16]	5	out[16]	10
y[16]	3	zr	0
z[16]	2	ng	0
zx	0		
nx	0		
zy	0		
ny	0		
f	1		
no	0		
zz	0		
nz	0		
fz	1		
noz	0		

Internal pins	
Name	Value
w[16]	8

HDL

```
// ALU3: ALU de 3 entradas (x, y, z)
CHIP ALU3 {
    IN
        x[16], y[16], z[16],
        zx, nx, zy, ny, f, no,
        zz, nz, fz, noz;
    OUT
        out[16], zr, ng;

    PARTS:
        // Etapa 1: combina x e y con ALU
        ALU(x=x, y=y, zx=zx, nx=nx,
            // Etapa 2: combina el resultado w con z
            // (w entra con zx=nx=false

```

3.6 Ejecución de los scripts de prueba

Para correr los casos de validación de las secciones 1.5 y 2.4 se usa File → Load Script, se escoge ALU32.tst o ALU3.tst, se pone Animate en "No animation" y se ejecuta todo el script con el botón de avance rápido (>>). Al terminar, la barra inferior del simulador muestra "End of script - Comparison ended successfully", lo que confirma que cada fila generada coincidió con el archivo .cmp correspondiente. Con View en "Output" se ve la tabla de resultados que produjo el chip.

Figura 5. ALU32.tst ejecutado completo (144 casos): Comparison ended successfully.

The screenshot shows the ALU32 test script execution. The HDL code defines the ALU32 chip with inputs xLow[16], xHigh[16], yLow[16], yHigh[16], and control signals zx, nx, zy, ny, f, no. The output pins are outLow[16], outHigh[16], zr, and ng. The test results table shows 144 cases, with the last row highlighted in yellow, indicating a successful comparison.

Name	Value	Name	Value
xLow[16]	0	outLow[16]	-1
xHigh[16]	-32768	outHigh[16]	-1
yLow[16]	-1	zr	0
yHigh[16]	-1	ng	1
zx	0		
nx	1		
zy	0		
ny	1		
f	0		
no	1		

End of script - Comparison ended successfully

Figura 6. ALU3.tst ejecutado completo (42 casos): Comparison ended successfully.

The screenshot shows the ALU3 test script execution. The HDL code defines the ALU3 chip with inputs x[16], y[16], z[16], and control signals zx, nx, zy, ny, f, no, zz, nz, fz, noz. The output pins are out[16], zr, and ng. The test results table shows 42 cases, with the last row highlighted in yellow, indicating a successful comparison.

Name	Value	Name	Value
x[16]	-8	out[16]	0
y[16]	12	zr	1
z[16]	3	ng	0
zx	1		
nx	0		
zy	1		
ny	0		
f	1		
no	0		
zz	1		
nz	0		
fz	1		
noz	0		

End of script - Comparison ended successfully

Video de sustentación

La sustentación en video de ambas preguntas está disponible en el siguiente enlace:

<https://youtu.be/jkqFG6NkcqQ>